CSE 144 Final Project Report
Transfer Learning Challenge
Group Members:
Sergio Quispe Vargas
Spring 2026

1 Introduction

The goal of this project was to classify images into 100 different classes for the project competition. This is a difficult task because the dataset is small compared with the number of classes. The training set has > 1000 images, which means each class has only a small number of examples. Because of this, training a large image model from the beginning would probably overfit and perform poorly.

Transfer learning was a good fit for this problem. Instead of starting with a blank model, we used models that were already trained on large image datasets. These models already know useful visual patterns such as edges, shapes, textures, and object parts. We then changed the final classification layer so the models could predict the 100 classes in this project.

The final approach used two pretrained models: EfficientNet-b3 and ViT-B/16. Both models were trained separately, and their predictions were combined at the end. This is an ensemble. The main reason for using two models was that different model types can make different mistakes. By combining them, the final prediction can become more stable and accurate.

An earlier idea was to use ConvNeXt Tiny as the main model. ConvNeXt Tiny is a strong convolutional neural network and would have been a reasonable transfer learning choice. However, the final notebook switched away from using only ConvNeXt Tiny and instead used two stronger and more complementary models. EfficientNet-B3 is efficient and strong for image classification, while Vit-b/16 uses a transformer based design that can learn image relationships in a different way. The Vit model also used SWAG pretrained weights, which made it especially strong for this small dataset. Because Vit-B/16 performed better on validation accuracy than EfficientNet-B3, the final ensemble gave more weight to Vit.

2 Dataset

The training folder contained 1000 images across 100 classes. The test folder contained 1,036 images.

The class labels were stored as folder names from 0 to 99. A custom dataset class called NumericImageFolder was used to make sure the folder labels were read numerically instead of alphabetically. This was important because alphabetical sorting can put labels in the wrong order. For example, without numeric sorting, folder 10 might be placed near folder 1 instead of after folder 9. The notebook checked that class 0 mapped to label 0, class 10 mapped to label 10, and class 99 mapped to label 99.

The training data was split into two parts for tuning. There were 863 images used for training and 216 images used for validation. The split was stratified, meaning it tried to keep the class balance similar in both sets.

For preprocessing, images were resized and cropped to 384 by 384 pixels. The images were converted into tensors and normalized using ImageNet mean and standard deviation values. This matched the type of preprocessing expected by pretrained models.

Several data augmentation techniques were used during training. These included random resized crops, random horizontal flips, small random vertical flips, random rotations, color changes, and

occasional grayscale conversion. These augmentations helped the model see slightly different versions of the same image, which reduced overfitting.

3 Implementation

3.1 Models

The final system used two models.

The first model was EfficientNet-B3. EfficientNet is designed to get strong image classification performance while keeping the model size reasonable. The notebook loaded EfficientNet-B3 with ImageNet pretrained weights. The original classifier was replaced with a new classifier for 100 classes. A dropout layer with probability 0.3 was added before the final linear layer. Dropout helps reduce overfitting by randomly turning off some features during training.

The second model was ViT-B/16, which stands for Vision Transformer Base with 16 by 16 image patches. Instead of processing images only with convolution layers, ViT divides an image into patches and uses transformer layers to learn relationships between those patches. The notebook used SWAG pretrained weights when available. These weights were stronger than normal ImageNet weights because they came from a larger pretraining setup. The original final head was replaced with a new linear layer for the 100 classes.

The switch from ConvNeXt Tiny to two models was an important design choice. ConvNeXt Tiny would have used one model architecture. The final approach used two different architectures: a convolution based model, EfficientNet-B3, and a transformer-based model, ViT-B/16. This gave the system two different ways of understanding images. EfficientNet can be strong at local image patterns, while ViT can capture broader relationships across image patches. Since the dataset was small, combining two pretrained models helped make the final predictions less dependent on the weaknesses of one model.

3.2 Training Strategy

The training was done in stages. This was one of the most important parts of the project.

In Stage 1, only the final classification head was trained. The pretrained backbone was frozen. This means most of the model weights were not changed. This is a safe first step because the new classifier learns the 100 project classes without damaging the useful features learned during pretraining.

In Stage 2, the last few layers of each model were unfrozen. For EfficientNet-B3, the last three feature blocks were trained along with the classifier. For ViT-B/16, the last three transformer blocks, the final layer norm, and the classification head were trained. This allowed the models to adjust some deeper features to the project dataset while keeping most pretrained knowledge.

In Stage 3, the best model from Stage 2 was trained again using the full training dataset. This used all 1,079 training images instead of only the 863-image training split. This helped the final model learn from as much data as possible before making test predictions.

The loss function was cross entropy loss with label smoothing set to 0.0. Cross entropy is a standard loss function for multi-class classification. Label smoothing was not used in the final version because the notebook notes that reducing it to 0.0 helped on this tiny dataset.

The optimizer was Adamw was used because it works well for deep learning models and includes weight decay, which helps reduce overfitting. The weight decay was set to 1e-4.

A cosine learning rate scheduler was used. This means the learning rate slowly decreased following a smooth curve during training. A high learning rate early in training helps the model learn faster, while a lower learning rate later helps it fine-tune more carefully.
Different learning rates were used for different parts of the model. This is called differential learning rates. The pretrained backbone used a smaller learning rate, while the new classifier head used a larger learning rate. This makes sense because the classifier head is new and needs to learn more, while the backbone already contains useful pretrained features and should be changed more carefully.
The batch size was 32. Training was done on CUDA, meaning a GPU was available.

4 Techniques Used

Several techniques were used to improve performance.
The first technique was transfer learning. Both EfficientNet-B3 and Vit-B/16 started from pretrained weights instead of random weights. This was helpful because the dataset was small.
The second technique was staged fine-tuning. The models were not fully trained all at once. First, only the head was trained. Then the last layers were unfrozen. Finally, the model was trained on the full dataset. This made training more stable.
The third technique was data augmentation. Random crops, flips, rotations, color changes, and grayscale changes made the training images more varied. This helped the model avoid memorizing the small training set.
The fourth technique was Mixup. combines two images together and mixes their labels. This encourages the model to make smoother decisions instead of becoming too confident.
The fifth technique was CutMix. cuts a patch from one image and places it into another image. The label is also mixed based on how much of each image is used. This helps the model learn from multiple objects or patterns in one training example.
The sixth technique was gradient clipping. The notebook clipped gradients to a maximum norm of 1.0. This helps prevent unstable updates during training.
The seventh technique was test-time augmentation, also called TTA. During prediction, each test image was transformed in 8 different ways. The model predicted each version, and the probabilities were added together. This made predictions more stable because the model did not rely on only one view of the image.
The eighth technique was assembling. The final prediction combined EfficientNet-B3 and ViT-B/16 probabilities. Since ViT-B/16 had better validation accuracy, the final ensemble used 60 percent ViT and 40 percent EfficientNet.

5 Experiments

The project compared two final backbones: EfficientNet-B3 and ViT-B/16.
EfficientNet-B3 was trained in three stages. In Stage 1, only the classifier head was trained for 10 epochs. This reached a best validation accuracy of 52.78 percent. In Stage 2, the last three blocks and classifier were trained with Mixup and CutMix for up to 20 epochs. The best validation accuracy improved to 68.06 percent. In Stage 3, the model was fine-tuned on the full training set for 15 epochs.
ViT-B/16 was also trained in three stages. In Stage 1, only the classification head was trained for 10 epochs. This reached a best validation accuracy of 75.00 percent. In Stage 2, the last three transformer blocks and head were trained with Mixup and CutMix. The best validation accuracy

improved to 79.63 percent. In Stage 3, the model was fine-tuned on the full training set for 15 epochs.
These experiments showed that ViT-B/16 was the stronger individual model. EfficientNet-B3 still helped because it provided a different type of model for the ensemble.

6 Results

EfficientNet-B3 reached a best validation accuracy of 68.06 percent.
ViT-B/16 reached a best validation accuracy of 79.63 percent.
Because ViT-B/16 performed better, the final ensemble used a higher weight for ViT. The ensemble weights were 0.6 for ViT-B/16 and 0.4 for EfficientNet-B3.
The final prediction file contained 1,036 predictions, one for each test image. The predicted labels covered the full label range from 0 to 99, which showed that the model was using all 100 possible classes.

7 Discussion

The best-performing part of the project was the ViT-B/16 model with SWAG pretrained weights. It performed much better than EfficientNet-B3 on validation accuracy. This suggests that the transformer model and stronger pretraining were very useful for this dataset.
The staged training strategy also worked well. Training only the head first gave the models a safe starting point. Unfreezing only the last layers later allowed the models to adapt without changing too many pretrained weights. This was important because the dataset was small.
Mixup and CutMix were useful because they made the training examples harder and more varied. The training accuracy sometimes looked lower during these stages because the labels and images were mixed. This does not necessarily mean the model was worse. It means the model was being trained on more challenging examples.
The ensemble was useful because it combined two different types of models. Even though ViT was stronger, EfficientNet could still add useful information. The final model used a 60 percent ViT and 40 percent EfficientNet weighting because the validation results showed that ViT deserved more influence.
One limitation is that the dataset was very small for a 100-class problem. Some classes may have had very few examples, which can make the model less reliable. Another limitation is that the notebook's later TTA validation cell showed a runtime error when rerun after some variables were missing. This likely happened because the notebook state was not fully rerun from the beginning. For reproducibility, the notebook should be run from the first cell to the last cell in order.
Future improvements could include testing ConvNeXt again as a third ensemble model, trying more ensemble weights, adding confusion matrix analysis, and checking which classes are most often confused. It may also help to test more pretrained models or use k-fold validation so every image gets used for validation once.

8 Reproducibility

The random seed was set to 42. The notebook set seeds for Python random, NumPy, PyTorch CPU, and PyTorch CUDA. It also set cuDNN deterministic behavior to make the results more repeatable.
The main libraries used were PyTorch, TorchVision, NumPy, pandas, PIL, tqdm, scikit-learn, and kagglehub.

To reproduce the work, the notebook should be run from top to bottom. First, it downloads the Kaggle dataset. Then it creates the numeric label mapping. Next, it builds the data loaders and models. After that, it trains EfficientNet-B3 and Vit-B/16 in three stages. Finally, it loads the saved checkpoints, runs test-time augmentation, combines the model probabilities, and saves submission.csv.
The final submission file was created with two columns: ID and Label. The ID column contained the test image filename, and the Label column contained the predicted class.

9 Team Contributions

I worked on dataset loading, numeric label mapping, preprocessing, and data loaders as well as model selection, EfficientNet-B3 training, ViT-B/16 training, staged fine tuning, worked on testtime augmentation, ensemble prediction, submission file creation, and report writing.

10 References

TorchVision documentation for EfficientNet-B3 pretrained models.
TorchVision documentation for ViT-B/16 pretrained models.
PyTorch documentation for AdamW, CrossEntropyLoss, and learning rate schedulers.